Practice Set: Base Class Pointers, Binding, and Virtual Functions

Section 1: Conceptual & Practical Questions (1–10)

Question 1: State the **Fundamental Rule of Pointer Assignment** across inheritance hierarchies in C++. Explain why assigning a base class pointer to a derived class object is legal and type-safe, while assigning a derived class pointer to a base class object is illegal without explicit casting.

Question 2: Differentiate between accessing class members via object variables using the dot operator (.) versus accessing class members through pointer variables using the arrow operator (->). Provide brief syntax examples for both.

Question 3: What is **Compile-Time (Early) Binding** in C++? Explain the core compiler rule that determines function call resolution when invoking non-virtual methods through pointers.

Question 4: Consider a non-virtual function show() in both Parent and Child classes. If Parent* p = &cobj; (where cobj is an instance of Child) is executed and p->show() is called, which class's show() function executes? Explain why this output occurs.

Question 5: Define a **Virtual Function** in C++. How does introducing the virtual keyword in a base class alter function call resolution from early binding to **Late (Run-Time) Binding**?

Question 6: Detail the decision-making mechanism used by the compiler when parsing a function call made through a base pointer for non-virtual versus virtual functions.

Question 7: Compare **Compile-Time / Early Binding** and **Run-Time / Late Binding** across definition, underlying resolution mechanism, keyword requirement, execution speed, and system flexibility.

Question 8: Explain the **Virtual Destructor Requirement** in base classes containing virtual functions. What severe runtime defect occurs if a derived class object created dynamically on the heap is deleted through a base class pointer when the base destructor is non-virtual?

Question 9: Write a complete C++ code demonstration showing **Compile-Time (Early) Binding**. Define a Parent class and a Child class with a non-virtual show() method. In main(), assign a Child object address to a Parent* pointer and call p->show().

Question 10: Write a complete C++ code demonstration showing **Run-Time (Late) Binding**. Modify the base class from Question 9 by marking show() as virtual. Demonstrate in main() how pointing Parent* p to a Parent object vs. a Child object dynamically calls the respective class implementations.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Can a base class pointer point to an object of any of its derived classes in C++?

Question 12 (Viva): Is a derived class pointer allowed to point directly to a base class object without explicit casting?

Question 13 (Viva): What C++ operator must be used when invoking a member function or variable through a pointer?

Question 14 (Viva): By default in C++, are function calls resolved based on the **pointer type** or the **actual object type** in memory?

Question 15 (Viva): What keyword must be placed in the base class function declaration to enable late binding?

Question 16 (Viva): Once a function is declared virtual in a base class, is it automatically virtual in all subsequent derived classes?

Question 17 (Viva): Why does Late Binding introduce a slight runtime execution overhead compared to Early Binding?

Question 18 (Viva): What internal table structure does the compiler generate to resolve virtual function addresses at runtime?

Question 19 (Viva): How is Runtime Polymorphism achieved in C++?

Question 20 (Viva): What syntax is recommended for defining a virtual destructor in a base class?

Answers & Explanations

Answer: 01

Fundamental Rule: A Base Class Pointer can point to an object of its Base Class OR to an object of ANY of its Descendant (Derived/Child) Classes. However, a Derived Class Pointer CANNOT point to a Base Class Object without explicit, unsafe casting.

Safety Explanation: A derived class object contains all inherited data attributes of the base class plus its own specialized members, making it type-safe for a base

pointer to interact with inherited base members. Conversely, a base class object lacks the specialized members defined in a derived class, so pointing a derived pointer to a base object would allow access to non-existent memory members, making it illegal.

Answer: 02

Dot Operator (.): Used to access class members directly when working with concrete object variables.

Example: Parent obj; obj.show();

Arrow Operator (->): Required when accessing class members or functions through a pointer variable.

Example: Parent* p = &obj; p->show();

Answer: 03

Compile-Time (Early) Binding: The default behavior in C++ where function calls made through pointers are linked to code blocks at compile time strictly based on the datatype of the pointer variable rather than the actual object stored at that address.

Core Compiler Rules (Without virtual):

Direct Object Calls: Decision is made based on the concrete **Object Type**.

Pointer Calls: Decision is made strictly based on the **Pointer Type**.

Answer: 04

Executed Function: Parent::show() will execute (e.g., printing "This is Parent class show()").

Explanation: Because show() is non-virtual, the compiler performs Compile-Time (Early) Binding. The compiler inspects only the type of pointer p (Parent*), ignoring the fact that p holds the address of a Child object in memory.

Answer: 05

Virtual Function: A member function declared inside a base class using the virtual keyword.

Shift to Late Binding: Marking a function virtual instructs the compiler to postpone function call binding until runtime (Late Binding). Resolution is then determined dynamically based on the actual target object stored in memory rather than the pointer type.

Answer: 06

If NOT Virtual: The compiler generates a direct assembly call/jump instruction to the base class function address at compile time.

If Marked virtual: The compiler generates dynamic lookup instructions to read the target object's virtual table (vtable) in memory at runtime to execute the corresponding overridden implementation.

Answer: 07

Parameter	**Compile-Time / Early Binding**	**Run-Time / Late Binding**
Definition	Function call is linked to code block at compile time.	Function call is linked to code block dynamically at runtime.
Mechanism	Resolved based on the **Pointer/Reference Datatype**.	Resolved based on the **Actual Object Type** in memory.
Keyword	Default behavior (no keyword needed).	Requires the **virtual** keyword in the base class.
Execution Speed	Faster execution (direct assembly call instruction).	Minimal overhead due to vtable pointer lookup.
Flexibility	Rigid and non-polymorphic.	Highly flexible; enables true Runtime Polymorphism.

Answer: 08

Virtual Destructor Requirement: Base classes containing virtual functions should always declare a virtual destructor (e.g., virtual ~Parent() = default;).

Runtime Defect: If a derived object allocated on the heap is deleted through a base pointer (delete p;) with a non-virtual destructor, only the base class destructor executes. The derived class destructor is bypassed, leaving derived attributes un-cleaned and causing memory leaks.

Answer: 09

```
#include <iostream>
using namespace std;
class Parent
{
  public:
    void show()
    {
```

```
        cout << "This is Parent class show()" << endl;
    }
};
class Child : public Parent
{
  public:
    void show()
    {
        cout << "This is Child class show()" << endl;
    }
};
int main()
{
    Child obj2;
    Parent* p = &obj2; // Base class pointer pointing to Child object

    // Early binding: Executed based on Pointer Type (Parent*)
    p->show(); // Output: This is Parent class show()

    return 0;
}
```

Answer: 10

```
#include <iostream>
using namespace std;
class Parent
{
  public:
    // virtual keyword enables Late Binding
    virtual void show()
    {
        cout << "This is Parent class show()" << endl;
    }
    virtual ~Parent() = default; // Virtual destructor best practice
};
class Child : public Parent
{
  public:
    void show() override
    {
        cout << "This is Child class show()" << endl;
    }
};
int main()
{
    Parent obj1;
```

```
    Child obj2;
    Parent* p;

    // Pointing to Parent Object
    p = &obj1;
    p->show(); // Output: This is Parent class show()

    // Pointing to Child Object
    p = &obj2;
    p->show(); // Output: This is Child class show()

    return 0;
}
```

Answer: 11
Yes, a base class pointer can legally point to its own class object or any descendant derived object.

Answer: 12
No, a derived class pointer cannot point to a base class object without explicit and unsafe casting.

Answer: 13
The **Arrow operator (->)**.

Answer: 14
By default, function calls are resolved based on the **pointer type** (Compile-time / Early Binding).

Answer: 15
The **virtual** keyword.

Answer: 16
Yes, once declared virtual in a base class, the function automatically remains virtual in all derived classes.

Answer: 17
Due to the slight runtime overhead of looking up the function address in the object's **vtable**.

Answer: 18
The **vtable (Virtual Table)**.

Answer: 19
By combining **Base Class Pointers** with **Virtual Functions**.

Answer: 20

virtual ~Parent() = default; (or explicit destructor definition prefixed with virtual).